

PRODUCT REQUIREMENT DOCUMENT

FLAM ASSIGNMENT

Fridge-to-Recipe Interactive Web Application

TARGET AUDIENCE

Bachelors / Students

PRIMARY STACK

React + Tailwind CSS

ARCHITECTURE

Serverless Backend Proxy

OUTPUT TYPE

Structured JSON Only

1. VISION & CORE OBJECTIVE

The **Fridge-to-Recipe Assistant** is an interactive single-page web application designed to help bachelors and busy individuals convert available fridge ingredients into executable, delicious recipes. Instead of functioning as a plain chatbot, the application ingests unformatted ingredient lists and converts them into structured JSON to power an interactive, step-by-step cooking interface.

Core Job To Be Done

"Transform my random list of fridge ingredients into a clear, bachelor-friendly recipe with step-by-step checkboxes, scalable portions, timing, and ingredient substitutions without chat overhead."

Key Non-Goal

This tool is explicitly **not a conversational chatbot**. The LLM response is never rendered as raw text paragraph streams, but strictly validated and mapped to typed React components.

2. ARCHITECTURE & DATA FLOW

To adhere to security best practices and assignment criteria, the system uses a dual-tier architecture. Client requests are dispatched to a lightweight Node.js/Serverless backend proxy holding the API key, which enforces strict JSON mode with the LLM API.

Component	Technology / Responsibility	Failure Prevention Mechanism
Frontend	React (Hooks), Tailwind CSS (Mobile-first styling, Desktop view target)	Defensive JSON parsing; Stale request guard via <code>useRef(requestId)</code> to ignore late responses.
Backend Proxy	Node.js / Express or Serverless Function	Holds API key out of browser client. Injects strict JSON prompt instructions before dispatching.
LLM Engine	Gemini / OpenAI / Groq API	Enforces JSON response mode and strict structural format without Markdown wrapper codeblocks.

3. REQUIRED STRUCTURED JSON SCHEMA

The backend proxy mandates that the AI return strictly valid JSON adhering to the following interface definition:

```
{
  "recipeId": "string (uuid)",
  "title": "string",
  "prepTimeMinutes": "number",
  "cookTimeMinutes": "number",
  "servings": "number",
  "difficulty": "Easy | Medium | Hard",
  "ingredients": [
    {
      "id": "string",
      "name": "string",
      "amount": "number",
      "unit": "string",
      "isEssential": "boolean",
```

4. FUNCTIONAL & INTERACTIVE REQUIREMENTS

The parsed JSON data feeds directly into dedicated React components providing an engaging, high-utility cooking session.

1. Ingredient Scaling & Swaps

- Dynamic Servings:** Scaling multiplier auto-recalculates ingredient quantities in real time.
- Substitutions View:** Tap any ingredient to see bachelor-friendly fallback options.
- Essential Flagging:** Highlights critical non-provided items missing from the prompt.

2. Interactive Step Checklist

- Checkable Steps:** Check off steps as cooking progresses with visual progress bar.
- Timing Indicators:** Individual step timers with clear duration tags (e.g., 5 mins).
- In-line Clarification:** Ask AI to clarify unclear steps ("sauté until golden") without full reset.

5. EXCEPTION HANDLING & RESILIENCY MATRIX

Failure Scenario	Detection Strategy	User-Facing UX Action
Malformed JSON	<code>JSON.parse()</code> exception caught in <code>validateResult.ts</code> .	Show clean error card. Allow single-click retry or option to tweak prompt text.
Missing Essential Fields	Schema validation check fails structural property inspection.	Reject rendering; route to <code>ErrorState</code> with specific guidance on missing data.
Slow Response / Timeout	Request latency exceeds threshold (>8 seconds).	Display friendly animated spinner; prevent duplicate submissions.
Out-of-Order Execution	Request ID ref mismatch upon API return.	Silently drop stale response; keep active UI focused on latest request.

6. SUCCESS CRITERIA & NON-FUNCTIONAL GUIDELINES

Non-Functional Target Checklist

Performance: Acceptable response window within 5-10s; instant local state updates for checkboxes and scaling.

Error Messaging: Minimal, crisp, actionable notifications — avoiding technical jargon or raw error dumps.

Responsiveness: Built with Tailwind CSS utility classes — fully responsive mobile-first, optimized for desktop demo.

Persistence: In-memory session state; zero database setup required for seamless submission evaluation.

7. EVALUATION & CRITICAL PATH INVARIANTS

- MUST NOT BREAK API Key Security:** Under no circumstances should the LLM API key be exposed inside browser network headers or bundle code.
- MUST NOT BREAK No Raw Chat Displays:** The UI must render structured React cards and interactive lists; direct raw text streams fail assignment requirements.
- Definition of Shipped:** User pastes random ingredients, receives a fully checkable recipe with timers and scalable ingredients, and can recover gracefully if model generation fails.

